

Hunter College, CUNY

Department of Computer Science

CSCI 39548: Practical Web Development

Building Modern Full-Stack Web Applications

Summer 2026

1 Details

- **Instructor Name:** Sourya Saha.
- **E-mail ID:** sourya.saha24@login.cuny.edu.
- **Modality:** Online-synchronous, conducted live over Zoom.
- **Office hours:** By appointment only, over email.
- **Credits:** 3.
- **Schedule:** Mondays & Wednesdays, 6:40pm to 9:48pm.
- **Start and End Date:** 1st June, 2026 to 8th July, 2026.

2 Course Description

CSCI 39548 Practical Web Development is a hands-on introduction to building and shipping modern full-stack web applications. The course progresses from the foundations of the modern web — version control with Git/GitHub, TypeScript-flavored JavaScript, and responsive layout with Tailwind CSS — into client-side application development with React 19, including component composition, hooks, client-side routing, and managing both client and server state. Students then build the backend half of an application using Node.js and Express, designing RESTful APIs and modeling relational data with the Prisma ORM against a PostgreSQL database. The course closes with authentication, deployment to cloud platforms, and a team final project. Emphasis throughout is on tools and practices used in industry today, on the discipline of writing readable code, and on collaborating effectively as a team.

3 Text and Reference Material

There is no required textbook for this course. Modern web development tooling changes rapidly, and the most accurate, up-to-date material lives in the official documentation. Students are expected to read and consult the following primary references throughout the term:

- **MDN Web Docs:** <https://developer.mozilla.org/> — canonical reference for HTML, CSS, and JavaScript.
- **React Documentation:** <https://react.dev/> — official React 19 docs and learning materials.
- **TypeScript Handbook:** <https://www.typescriptlang.org/docs/>.
- **Tailwind CSS Docs:** <https://tailwindcss.com/docs>.

- **TanStack Query Docs:** <https://tanstack.com/query/latest>.
- **React Router Docs:** <https://reactrouter.com/>.
- **Zustand Docs:** <https://zustand.docs.pmnd.rs/>.
- **Express Docs:** <https://expressjs.com/>.
- **Prisma Docs:** <https://www.prisma.io/docs>.

Any additional high-quality online materials — tutorials, blog posts, video courses, or open-source repositories — are welcome and encouraged as supplementary learning. Students are responsible for evaluating the recency and accuracy of any third-party material they choose to study from.

4 Course Requirements

Students must have completed **CSCI 235** before taking this course. It is strongly recommended that students are comfortable writing, running, and debugging code on their own, and have at least basic exposure to git/GitHub, JavaScript, HTML, and CSS prior to the first class. Students should also have a personal computer (macOS, Windows, or Linux) on which they can install development tools, and a reliable internet connection adequate for live video sessions.

5 Course Objectives

By the end of the course, students should be able to-

- **CO1:** Apply collaborative software development workflows using Git and GitHub, including branching, pull requests, and code review.
- **CO2:** Build responsive, accessible web user interfaces using semantic HTML, modern CSS through Tailwind utility classes, and TypeScript fundamentals.
- **CO3:** Develop interactive single-page applications using React 19 with functional components, hooks, and component composition patterns.
- **CO4:** Manage client state with Zustand and server state with TanStack Query, handling asynchronous data flows and API integration.
- **CO5:** Implement client-side routing and multi-page navigation in single-page applications using React Router.
- **CO6:** Design and implement RESTful backend services with Node.js and Express, including middleware, request validation, and CORS configuration.
- **CO7:** Model relational data with the Prisma ORM and PostgreSQL, perform CRUD operations, and apply schema migrations.
- **CO8:** Deploy full-stack applications to production using Vercel, Render, and Neon, and integrate third-party authentication services.

6 Course Expectations

- This is an **online-synchronous** course delivered live over Zoom. Students are expected to attend every class session on time and check **Brightspace** regularly for announcements, lecture material, assignments, and the exit assessment, unless otherwise specified.
- Students are expected to have their development environment installed and working by the end of Class 1 and to bring it to every subsequent session.

- Classes will mix slide-based discussion with live coding. Students are expected to follow along, ask questions, and participate in coding exercises during class.
- The instructor can be emailed to schedule office hour appointments.
- This course will use Brightspace for announcements, lecture slides, assignment distribution and submission, the final project, and the exit assessment.

7 Exams and Assignments:

- **Lectures:** Each class will combine slide-based discussion of concepts with live code demonstrations conducted online. Students are strongly encouraged to follow along and write code in real time during class, as the live coding portions are where the most practical learning occurs.
- **Assignments:** There will be four individual assignments throughout the term, released and submitted via Brightspace. The specific deliverables for each assignment will be communicated separately. Assignments are due before class on the dates announced. It is the student's responsibility to track due dates.
- **Final Project:** A group full-stack web application, completed in teams of **two to three** students. Solo final projects are not permitted. Teams will design, build, and deploy a working application using the course's tech stack, and present a live demo during the final class. Details and rubric will be released separately.
- **Exit Assessment:** A one-hour proctored assessment administered through Brightspace at the end of the term. Students must score **at least 60%** on the exit assessment in order to pass the course, regardless of their cumulative course grade.

8 Course Policies

- Assignments must be submitted on time. Late submissions will not generally be accepted, because feedback and solutions may be discussed before the next module begins.
- Assignments may be accepted up to one week late if the instructor is contacted prior to the original submission date and finds the reason viable. Late submissions are subject to up to a **20% point reduction per day**. A cascade of late submissions caused by an earlier late submission will not be accepted.
- The exit assessment must be completed on the scheduled date. For students with verified reasons to be absent, a makeup will be arranged on an individual basis. The instructor must be contacted at least one week prior to the scheduled date; makeups cannot be arranged after the fact.
- The final project is a group assignment of two to three students. Solo final projects are **not permitted** under any circumstances. Teams must be formed and registered with the instructor by the deadline announced in class. Individual contribution within the group will be considered when assigning the final project grade.
- **Use of AI:** Students are **welcome to use AI tools** (e.g., ChatGPT, Claude, GitHub Copilot, Cursor) on all assignments and on the final project. Modern web development is increasingly done alongside AI assistants, and learning to use them well is part of becoming a capable developer. The expectation is **informed use**: students should use AI to understand code, explore alternatives, debug, and deepen their understanding of the underlying concepts — not to generate code they cannot read, explain, or modify. If asked in class, in office hours, or during code review, students must be able to walk through any code they submit, line by line, in their own words. AI use is **prohibited during the proctored exit assessment**.
- **Learning to get things right takes practice!** Students are encouraged to build small side projects, work through tutorials, and read other people's code outside of course assignments. Time spent writing code on your own keyboard is the single highest-leverage thing you can do in this course.

9 Grading Schema

Assignment 1	10%
Assignment 2	10%
Assignment 3	10%
Assignment 4	20%
Final Project (group)	40%
Exit Assessment	10%

Students must score **at least 60%** on the exit assessment in order to pass the course. Overall course grading will follow the Hunter College standard in the *Grade Definitions and Grading Policy*:

<https://hunter-undergraduate.catalog.cuny.edu/policies-and-requirements/academic-requirements/grading-policy/grading-definition>

10 Course Modules

The following are the modules included in this course. Each week contains two synchronous class sessions (Monday and Wednesday). Topics may be adjusted based on class progress.

Week	Unit	Topics
1	Foundations	Course Introduction; Dev Environment Setup (Node, VS Code, terminal); Git & GitHub Collaboration Workflow; Branches, Pull Requests, Code Review; Modern JavaScript Essentials (<code>const/let</code> , arrow functions, array methods, destructuring, spread/rest, modules); Introduction to TypeScript (types, interfaces, generics overview)
2	From the DOM to React	Asynchronous JavaScript (Promises, <code>async/await</code> , <code>fetch</code>); DOM Basics and Event Handling; Why React Exists; Vite Project Bootstrapping; First React Component; Props, <code>useState</code> , <code>useEffect</code> ; Rendering Lists; Conditional Rendering; Forms in React
3	UI Patterns and Data Fetching	Component Composition; Lifting State Up; Controlled vs Uncontrolled Inputs; Tailwind CSS Fundamentals (utility-first styling, layout, responsive design); Server State with TanStack Query; <code>useQuery</code> , <code>useMutation</code> ; Cache Invalidation and Stale Time
4	Routing and Client State	Client-Side Routing with React Router; <code>Routes</code> , Route Parameters, Nested Routes, <code>Outlet</code> ; Navigation and Links; Client State Management with Zustand; Stores, Selectors, Persistence; When to Use Client State vs Server State
5	Backend Development	Node.js Runtime Overview; Building APIs with Express; Routing, Middleware, Error Handling; REST Conventions; CORS; Request Validation; Modeling Relational Data with Prisma ORM; PostgreSQL Basics; Schema, Migrations, Relations; CRUD Operations from Express to Prisma to Postgres
6	Auth, Deployment, and Final Demos	Authentication with Clerk; Protected Routes and Authenticated API Requests; Deploying the Frontend to Vercel; Deploying the Backend to Render; Hosting PostgreSQL on Neon; Connecting It All in Production; Exit Assessment Review; Final Project Group Demos

Note: Topics may be adjusted based on class progress. There are no scheduled CUNY holidays falling on a Monday or Wednesday within the term window (June 1 – July 9, 2026).

11 Hunter College Policies

11.1 Disabilities And Accommodations

The support for academic adjustments provided by the Office of AccessABILITY are based upon current supporting medical, educational and/or psychological documentation. All documents are kept confidential and will only be used for support and accommodation purposes. For more information, please visit the following webpage:
<https://hunter.cuny.edu/students/health-wellness/accessibility/students/>

11.2 Academic Integrity

<https://hunter.cuny.edu/students/student-affairs/office-of-student-conduct/academic-integrity/>

Academic dishonesty is prohibited in The City University of New York. Penalties for academic dishonesty include academic sanctions, such as failing or otherwise reduced grades, and/or disciplinary sanctions, including suspension or expulsion.

11.3 Definitions and Examples of Academic Dishonesty

Cheating is the unauthorized use or attempted use of material, information, notes, study aids, devices or communication during an academic exercise. The following are some examples of cheating, but by no means is it an exhaustive list:

- Copying from another student during an examination or allowing another to copy your work.
- Unauthorized collaboration on a take-home assignment or examination.
- Taking an examination for another student, or asking or allowing another student to take an examination for you.
- Allowing others to research and write assigned papers or do assigned projects, including use of commercial term paper services.
- Giving assistance to acts of academic misconduct/dishonesty.
- Fabricating data (all or in part).
- Submitting someone else's work as your own.

Plagiarism is the act of presenting another person's ideas, research or writings as your own. The following are some examples of plagiarism, but by no means is it an exhaustive list:

- Copying another person's actual words without the use of quotation marks and footnotes attributing the words to their source.
- Presenting another person's ideas or theories in your own words without acknowledging the source.
- Using information that is not common knowledge without acknowledging the sources.
- Failing to acknowledge collaborators on homework and laboratory assignments.
- Internet plagiarism includes submitting downloaded term papers or parts of term papers, paraphrasing or copying information from the Internet without citing the source, and "cutting and pasting" from various sources without proper attribution.

11.4 INC - Incomplete

<https://hunter-undergraduate.catalog.cuny.edu/policies-and-requirements/academic-requirements/course-policies/incomplete-work-in-a-course>

When a student for valid reason does not complete the work assigned in a course (including the final exam, papers, etc.) and in the view of the instructor still has a reasonable chance to pass the course, the student shall be given the grade INC (incomplete). The student must explain the reason to the instructor or, in the absence of the instructor, to the department chair and arrange a schedule for making up the missing coursework. These steps must be taken

as soon as possible and no later than the end of the second week of the following semester. The student shall then be given the opportunity to complete the course without penalty beyond previously established penalties for lateness.

The length of time permitted for completing missing coursework remains at the discretion of the instructor and shall be indicated in writing to the student, but shall not extend beyond the end of the semester following the one in which the course was taken. Unless the student submits the work by the date specified by the instructor, the grade will automatically become FIN on the student's permanent record. (Under certain circumstances, where the student must repeat class sessions or laboratories in a course not given during the following semester, the FIN grade may later be converted to the appropriate letter grade.)

If the faculty member wishes to extend the deadline for the student to complete the coursework beyond one semester, the faculty member and the student must enter into a written contract clearly specifying the deadline. This contract must be written during the semester following the one in which the course was taken. The student must be aware that the INC grade will change to a FIN grade until the work is completed. The written contract must accompany the change of grade form. If a student has not filed a contract with the faculty member but still wishes to complete the work and have a FIN grade changed, the student can appeal to the Senate Grade Appeals Committee. The appeal must include the reason for failing to complete the work and must be accompanied by a supporting letter from the faculty member who issued the INC grade or, if the faculty member is no longer at the college, from the department chair. Appeals with no endorsement will be denied.

11.5 Grade Appeal Process

<https://hunter-graduate.catalog.cuny.edu/policies/appeals-grades>

The grade appeal process is designed to adjudicate the fairness of the final grade a student receives in a course. The appeals committees (see below) cannot alter the grade given by the instructor on a particular assignment. The appeals committee shall consider whether the instructor followed the grading criteria laid out in the course syllabus and whether the grading was arbitrary and capricious. (On occasion, an instructor may have good reason to alter a syllabus; in such cases, students must be informed in writing before the changes take effect.) A grade appeal should be limited to the grade that the student received. Other complaints about a course and/or an instructor should be raised in a timely manner (during the semester in which the course is given) to the department chair or program head.

Please visit webpage linked above for additional information about the process.